

# Assignment: Building Resilient Distributed Systems

Course: Parallel and Distributed Computing (PDC)

Total Points: 100

## Overview

For this assignment we are moving away from purely theoretical concepts and looking at a real-world scenario. Modern full-stack web applications are, at their core, distributed systems. When independent components like frontends, backends, databases, and external APIs talk to each other over a network, things break.

Your goal here is to analyze a baseline application, figure out why it fails under distributed conditions, design some architectural fixes, and actually write the code to solve one of those problems.

## Reference Material

We will be referencing the architecture from the video How To Build a FastAPI and React Full Stack App by Tech With Tim.

Link: <https://www.youtube.com/watch?v=13tMEW8r6C0>

Note: You can use his codebase as a starting point, or spin up your own minimal FastAPI mock to simulate what he built.

## The Scenario: StudySync is Crashing

Imagine you just got hired as a backend engineer at an ed-tech startup called StudySync. The team built an MVP using the exact FastAPI and React stack from the video. They use Clerk for authentication and webhooks, a standard database, and they make calls to an external LLM API.

They just launched to 1,000 beta users, and the bug reports are flooding in. Specifically, you have been tasked with looking into these three major issues:

- Problem 1 (Synchronization): Two users try to edit the same shared document at the exact same time. The app just silently overwrites one user's changes with the other's. This is the classic Lost Update anomaly.
- Problem 2 (Coordination): When a user cancels their premium subscription via Clerk, a webhook is sent to our backend. But if our network blips and drops that webhook, the user stays premium forever. The systems are out of sync.
- Problem 3 (Fault Tolerance): Sometimes the external LLM API goes down and takes 60 seconds to timeout. Because FastAPI is waiting synchronously, the server thread blocks, and suddenly the entire application hangs for everyone.

## What You Need To Do

Here is how I want you to tackle this. The assignment is broken down into three parts.

## Part 1: Analyze the Mess (Written Report)

Write a brief analysis of about a page identifying the root cause of the three problems in the current naive architecture.

- Where exactly do the synchronization issues occur in the database and API lifecycle?
- How do Clerk webhooks currently handle coordination, and why does a dropped event cause permanent state inconsistency?
- Why is a synchronous LLM call acting as a single point of failure here?

## Part 2: Design a Better System (Written Report and Diagram)

Put on your software architect hat. Propose technical solutions for all three problems.

- Sync: Design an Optimistic Locking approach, like versioning, or Operational Transformation. Please include a UML Sequence Diagram showing how your system handles two concurrent users.
- Coordination: Design a fault-tolerant webhook handler. Explain how you would use idempotency keys and retries, or a dead-letter queue, so we never drop a cancellation again.
- Fault Tolerance: Design a Circuit Breaker pattern with a Fallback response for the LLM.
- Trade-offs: Somewhere in this section, give me a quick discussion on the CAP theorem trade-offs you are making regarding Consistency versus Availability versus Latency.

Keep Parts 1 and 2 concise. Your final PDF should be 3 pages maximum.

## Part 3: Fix It (Code and Demo)

You only need to write code for ONE of the problems. Pick whichever of the three you find most interesting.

- Implement a minimal, working fix in the FastAPI codebase.
- Write a script or test case that simulates the failure, like mocking the LLM crashing or firing two concurrent updates, and proves your code handles it gracefully.
- Record a quick screen-share video of 2 minutes maximum. Show me the system failing without your code, and then succeeding with your code.

## Strict Submission Rules

To make grading fair and to ensure everyone is writing their own code, you must follow these trademark rules. If you miss the custom header requirement, you will automatically get a zero for Part 3. No exceptions.

1. Custom Middleware Header: You must write a FastAPI middleware so that every single API response returns a custom HTTP header formatted exactly as X-Student-ID: [Your-Actual-Student-ID].
2. GitHub Repo Naming: Name your repository exactly PDC-Sp24-[Your-ID]-[Your-LastName].

3. README: The very first line of your README.md file must be your Name and Student ID. Include instructions on how I can run your code and tests.

What to submit to the classroom:

- Your PDF Report: Containing Part 1 and Part 2.
- Your GitHub Link: Paste this in the submission text box.
- Your Demo Video: An mp4 upload or an unlisted YouTube or Loom link.

## How You Will Be Graded

Here is exactly what the TAs and I will be looking for when we grade your work.

### Part 1: Analysis (20 Points)

You accurately identify the root cause of the Sync bug, the Coordination bug, and the Fault Tolerance bug using proper terminology.

### Part 2: Design (30 Points)

You propose a viable architecture fix for all three problems. You explain the CAP theorem trade-offs well. You include a readable, logically sound sequence diagram for the sync fix.

### Part 3: Code (25 Points)

Your code actually implements a distributed pattern like a Circuit Breaker, Queue, or Locking. Your test case successfully triggers the failure state. Your code is readable. And most importantly, you include the X-Student-ID header.

### Part 3: Demo (15 Points)

Your 2-minute video clearly demonstrates the before and after states of your chosen bug.

### Formatting (10 Points)

You followed the repository naming rules, included README instructions, and your PDF is within the 3-page limit.

Good luck. Have fun with this, as these are the exact types of problems you will be solving on a daily basis as a backend engineer. Reach out during office hours if you get stuck.